Programação Orientada a Objetos

Aula 07 - Mais de POO em python:

Heranças múltiplas, classes abstratas, modificadores de acesso e sobrecarga de operadores

Prof. Dr. Eduardo de Olivindo (eduardo.olivindo@ifce.edu.br)

Tecnologia em Análise e Desenvolvimento de Sistemas

IFCE - Campus Itapipoca



Objetivos da aula

- Compreender quando usar heranças múltiplas em Python.
- Aplicar classes abstratas para definir contratos de implementação.
- Entender como modificadores de acesso se comportam em hieraquias.
- Implementar sobrecarga de operadores com métodos especiais.
- Integrar os quatro temas em exemplos práticos de modelagem OO.

Roteiro

- Parte 1: Heranças múltiplas
- Parte 2: Classes abstratas
- Parte 3: Modificadores de acesso na herança
- Parte 4: Sobrecarga de operadores
- Parte 5: Exercício integrador

Parte 1: Heranças múltiplas

O que é herança múltipla?

- Uma classe pode herdar de mais de uma superclasse ao mesmo tempo.
- Isso permite combinar comportamentos de classes diferentes.
- Em Python, a busca por métodos segue a MRO (Method Resolution Order).
- É útil quando a classe representa mais de um papel no domínio.

Exemplo de domínio

- `Impressora` sabe imprimir.
- `Scanner` sabe digitalizar.
- `Multifuncional` herda os dois comportamentos.

```
class Impressora:
    def imprimir(self, texto):
        return f"Imprimindo: {texto}"

class Scanner:
    def digitalizar(self):
        return "Digitalizando documento..."

class Multifuncional(Impressora, Scanner):
    pass
```

Testando herança múltipla

```
equipamento = Multifuncional()
print(equipamento.imprimir("Relatório"))
print(equipamento.digitalizar())
```

- O objeto reutiliza comportamentos de duas classes.
- Não há duplicação de código.
- O cuidado principal está em conflitos de métodos.

Conflito de métodos e MRO

```
class A:
    def acao(self):
        return "Ação em A"

class B:
    def acao(self):
        return "Ação em B"

class C(A, B):
    pass

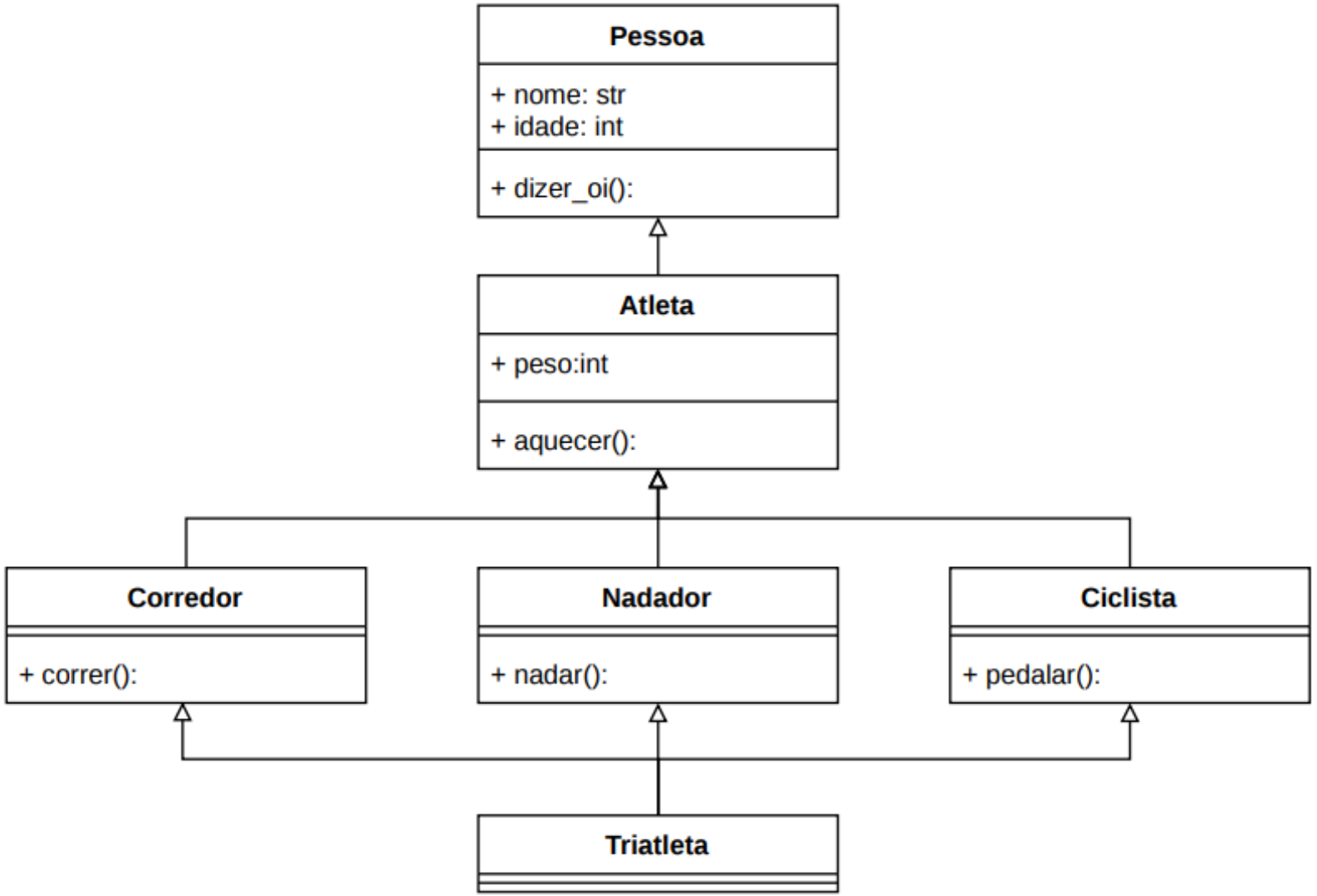
print(C().acao())
print(C.__mro__)
```

- `C().acao()` usa `A.acao()` porque `A` aparece antes de `B`.
- `__mro__` mostra a ordem de resolução de métodos.

Boas práticas

- Prefira classes pequenas com responsabilidade únicas.
- Evite duplicar atributos equivalentes em múltiplas bases.
- Documente a ordem das classes-base quando ela for relevante.
- Se houver acoplamento excessivo, considere composição.

Exercício 1: Implemente, teste e avalie o MRO



Parte 2: Classes abstratas

O que é classe abstrata?

- Define um contrato comum para subclasses.
- Não deve ser instanciada diretamente.
- Pode conter métodos concretos e métodos abstratos.
- Em Python, usamos o módulo `abc`.

Classe abstrata em Python

```python
from abc import ABC, abstractmethod

class Forma(ABC):
    @abstractmethod
    def area(self):
        pass

class Retangulo(Forma):
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

    def area(self):
        return self.base * self.altura
```

Testando classe abstrata

- `Forma` não pode ser instanciada.
- `Retangulo` só é válido porque implementa `area()`.

```
figura = Retangulo(3, 4)
print(figura.area())  # 12

# erro:
# f = Forma()
```

- O contrato abstrato padroniza implementações.
- Isso favorece polimorfismo com segurança.

Exercício 2

- A classe abstrata FiguraGeomética deve ter mais um método abstrato perimetro().
- Além de Retângulo, crie as classes concretas Círculo e Triângulo.
- Teste as classes e Objetos criados

Exercício 3

- A classe Jogador do exercício da lista sobre herança denominado `Jogo Pedra, Papel, Tesoura` deve ser abstrata? se sim, adapte seu programa para garantir este novo formato.

Parte 3: Modificadores de acesso na herança

Relembrando (visto em aula anterior)

- Público: `atributo`
- Protegido (convenção): `_atributo`
- Privado (name mangling): `__atributo`
- Hoje o foco é: **o que acontece com cada nível quando há herança?**

O problema: herdar e não conseguir acessar

```
class ContaBancaria:
    def __init__(self, titular, saldo_inicial):
        self.titular = titular          # público
        self._agencia = "0001"          # protegido
        self.__saldo = saldo_inicial    # privado (name mangling)

class ContaPoupanca(ContaBancaria):
    def resumo(self):
        print(self.titular)     # ok
        print(self._agencia)    # ok
        print(self.__saldo)     # AttributeError!
```

- A classe filha herda os atributos, mas nem todos ficam acessíveis.

Por que `__saldo` falha na filha?

- O *name mangling* renomeia `__saldo` para `_ContaBancaria__saldo`.
- Dentro de `ContaPoupanca`, `self.__saldo` é procurado como `_ContaPoupanca__saldo`.
- Esse atributo não existe: só existe `_ContaBancaria__saldo`.
- Ou seja, atributos com `__` **não são projetados para serem usados pelas subclasses**.

```
print(vars(ContaBancaria(...)))
# {'titular': ..., '_agencia': '0001', '_ContaBancaria__saldo': ...}
```

Solução: use `_` (protegido)

```
class ContaBancaria:
    def __init__(self, titular, saldo_inicial):
        self.titular = titular
        self._agencia = "0001"
        self._saldo = saldo_inicial     # agora protegido, não privado

class ContaPoupanca(ContaBancaria):
    def render_juros(self, taxa):
        self._saldo += self._saldo * taxa   # acesso direto funciona
        return self._saldo
```

- `_atributo` é acessível em qualquer subclasse, por convenção.
- `__atributo` só deve ser usado para detalhes internos da própria classe, sem intenção de herança.

Resumo comparativo

Modificador	Acesso na própria classe	Acesso na classe filha	Acesso fora das classes
`atributo` (público)	Sim	Sim	Sim
`_atributo` (protegido)	Sim	Sim (por convenção)	Sim (não recomendado)
`__atributo` (privado)	Sim	Não	Não

- Quando a subclasse precisa **ler ou alterar** o atributo, prefira `_atributo`.
- Quando o atributo é **detalhe de implementação** que a subclasse não deve tocar, use `__atributo`.

Se realmente precisar expor o privado à filha

```
class ContaBancaria:
    def __init__(self, titular, saldo_inicial):
        self.titular = titular
        self.__saldo = saldo_inicial

    def _get_saldo(self):     # método protegido "ponte"
        return self.__saldo

class ContaPoupanca(ContaBancaria):
    def resumo(self):
        return self._get_saldo()   # acesso controlado, sem herdar o atributo direto
```

- A classe base mantém o encapsulamento e ainda assim libera acesso via método.

Exercício 4

- Crie `Veiculo` com `self.marca` (público), `self._km_rodado` (protegido) e `self.__numero_chassi` (privado).
- Crie `Carro(Veiculo)` com um método `detalhes()` que tenta imprimir os três atributos.
- Mostre qual linha causa erro e explique o porquê usando name mangling.
- Corrija o código trocando o atributo necessário de `__` para `_`.

Parte 4: Sobrecarga de operadores

O que é sobrecarga de operadores?

- Em Python, usamos métodos especiais para redefine operadores para objetos de uma classe.
- Exemplos comuns:
  - `__add__` para `+`
  - `__sub__` para `-`
  - `__eq__` para `==`
  - `__lt__` para `<`
  - `__str__` para representação textual
- Outros *dunders* possíveis: `__mul__`, `__truedivi__`, `__floordiv__`,, `__pow__`, `__len__`, `__hash__`, `__le__`, `__ge__`, `__ne__` ...

Exemplo: classe Vetor2D

```
class Vetor2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, outro):
        return Vetor2D(self.x + outro.x, self.y + outro.y)

    def __eq__(self, outro):
        return self.x == outro.x and self.y == outro.y

    def __str__(self):
        return f"({self.x}, {self.y})"
```

Teste de operadores

```
v1 = Vetor2D(2, 3)
v2 = Vetor2D(4, 1)

v3 = v1 + v2
print(v3)                    # (6, 4)
print(v3 == Vetor2D(6, 4)) # True
```

- O uso fica mais natural para quem consome a classe.
- A semântica do operador deve ser intuitiva.

Outro exemplo

```
class Aluno:
    def __init__(self, nome, media):
        self.nome = nome
        self.media = media

    def __lt__(self, outro):
        return self.media < outro.media

    def __repr__(self):
        return f"Aluno({self.nome}, {self.media})"

turma = [Aluno("Ana", 8.5), Aluno("Beto", 7.0), Aluno("Caio", 9.1)]
print(sorted(turma))
```

Cuidados com sobrecarga

- Implemente apenas operadores que façam sentido no domínio.
- Mantenha consistência entre igualdade e ordenação.
- Evite comportamentos surpreendentes.
- Sempre documente regras de comparação e combinação.

Exercício 5

- Crie uma classe `Tempo` com `horas` e `minutos`.
- Sobrecarregue `+` para somar tempos.
- Sobrecarregue `==` para comparar igualdade.
- Implemente `__str__` no formato `HH:MM`.

Parte 5: Exercício integrador

Exercício 6: Desafio final

- Modele um sistema de formas geométricas com:
  - classe abstrata `Forma` com `area()`
  - `Retangulo` e `Circulo`
- Crie uma classe `Colorivel` com atirbuto `cor` e método `alterar_cor()`.
- Crie `RetanguloColorido` usando herança múltipla.
- Use atributo protegido para armazenar a cor.
- Sobrecarregue os métodos necessários para comparar as formas (>, <, ==).

Fechamento

- Herança múltipla combina papéis e comportamentos.
- Classes abstratas definem contratos claros.
- Use `_atributo` quando a subclasse precisa acessar; `__atributo` fica restrito à própria classe.
- Sobrecarga de operadores melhora legibilidade e expressividade.
- Um bom modelo OO equilibra clareza, reuso e semântica.